



**Intelligence Academy**

**Scikit-Learn 101**

**02. Introduction: Why a unified API matters**

**Mejbah Ahammad**

## What "Unified API" Means

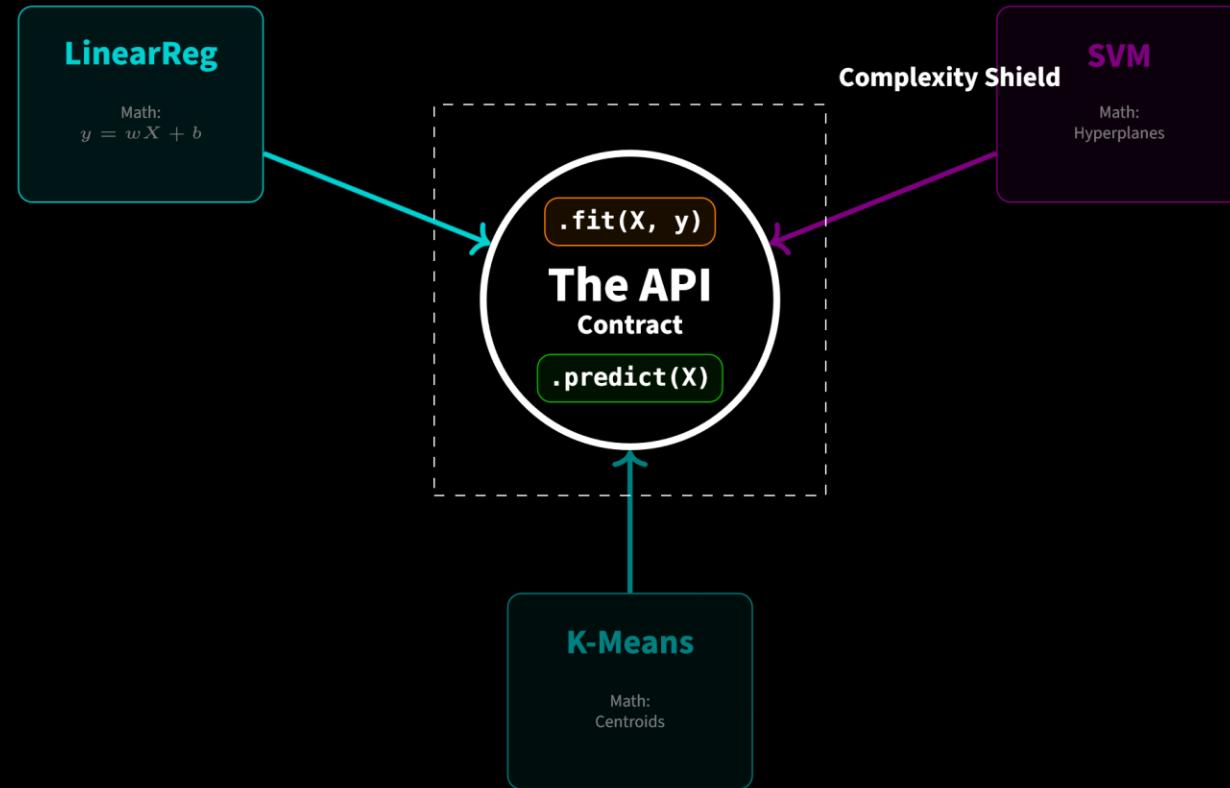
### 1.1 The Philosophy of Consistency

In many libraries, every algorithm has a different name for training ('train', 'learn', 'optimize').

In scikit-learn, there is only **One Language**.

Whether you are doing Regression, Classification, or Clustering, the commands are identical.

This drastically reduces the cognitive load on the data scientist.



## 1.2 The Three Fundamental Verbs

The Unified API is built on three core actions. If you know these, you know 90% of the library.

### 1. Estimators



**The Learners**  
Builds internal state  
from data.

```
.fit(X, y)
```

### 2. Transformers



**The Modifiers**  
Changes the representation  
of data.

```
.transform(X)
```

### 3. Predictors



**The Oracles**  
Generates predictions  
for new data.

```
.predict(X)
```

## 1.3 The Power of Swappability

Because the API is unified, you can switch from a simple model to a complex one by changing **only one line of code**. The rest of your pipeline remains untouched.

This code  
**NEVER**  
changes

```
from sklearn import ...
```

```
model = LinearRegression() > RandomForest()
```

Swap Here

```
model.fit(X_train, y_train)
preds = model.predict(X_test)
score = model.score(X_test, y_test)
```

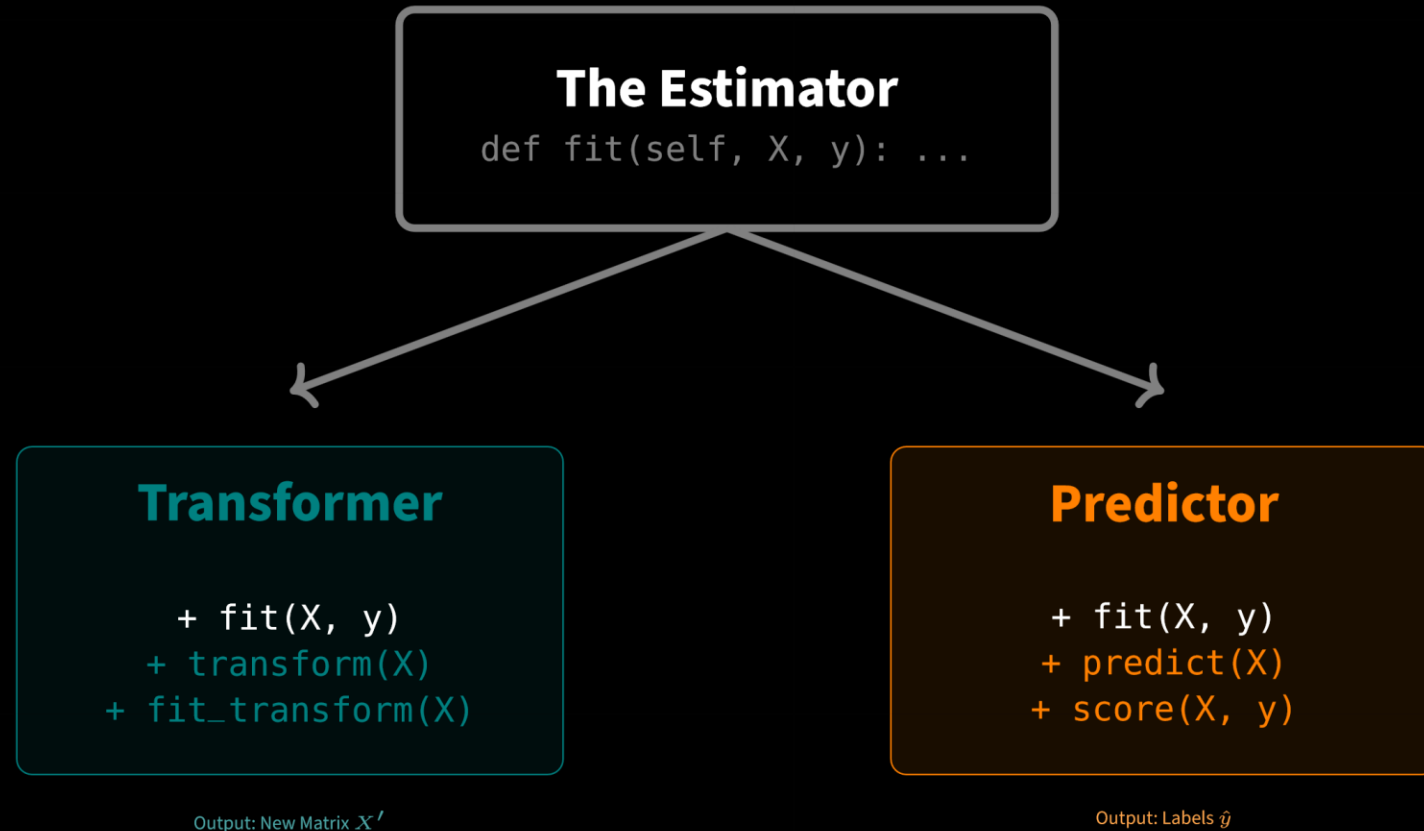


## The Shared Interface: Taxonomy

### 2.1 The Interface Hierarchy

Think of `Estimator` as the parent species.

- **Base Estimator:** The ancestor. It knows how to learn ('fit').
- **Transformer:** A descendant that learns to *modify* data (e.g., Scaler, PCA).
- **Predictor:** A descendant that learns to *judge* data (e.g., Classifier, Regressor).



## 2.2 Visualizing the Data Flow

What goes in? What comes out?

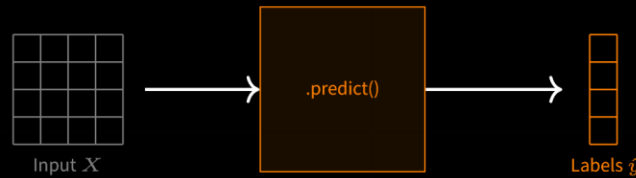
- **Transformer:** Preserves the structure (Matrix in  $\rightarrow$  Matrix out).
- **Predictor:** Collapses the structure (Matrix in  $\rightarrow$  Vector out).

### A. Transformer Flow



---

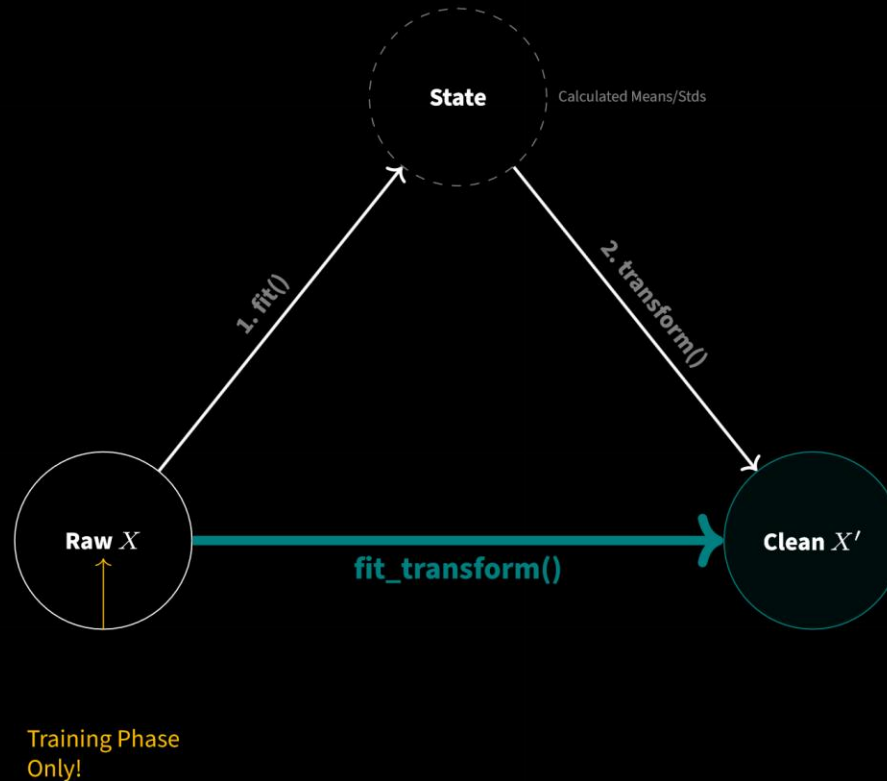
### B. Predictor Flow





## 2.3 The Efficiency Shortcut

Why call two methods when you can call one? `fit_transform(X)` is computationally efficient and prevents code redundancy during training.



## The Lifecycle: Fit → Score

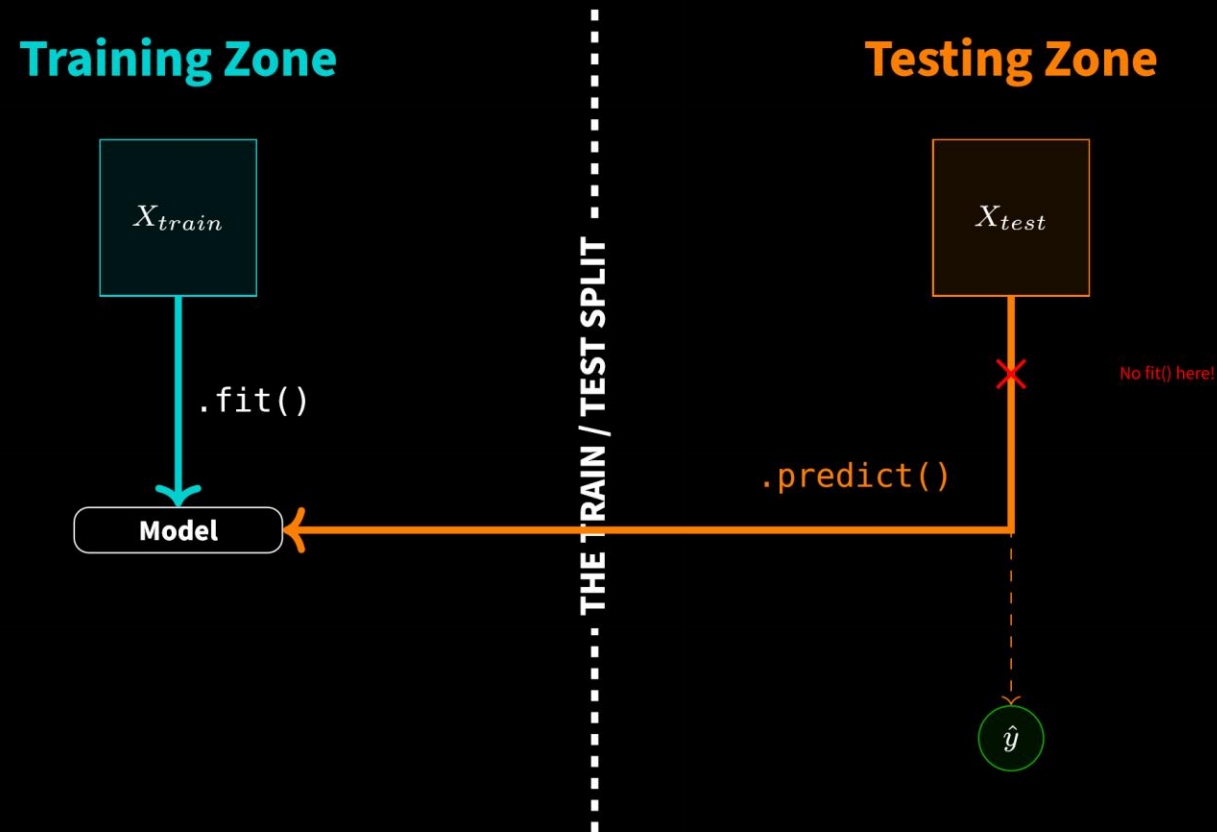
### 3.1 The Standard Sequence

Every supervised learning task follows this linear progression. First we learn (Fit), then we apply (Predict), then we judge (Score).



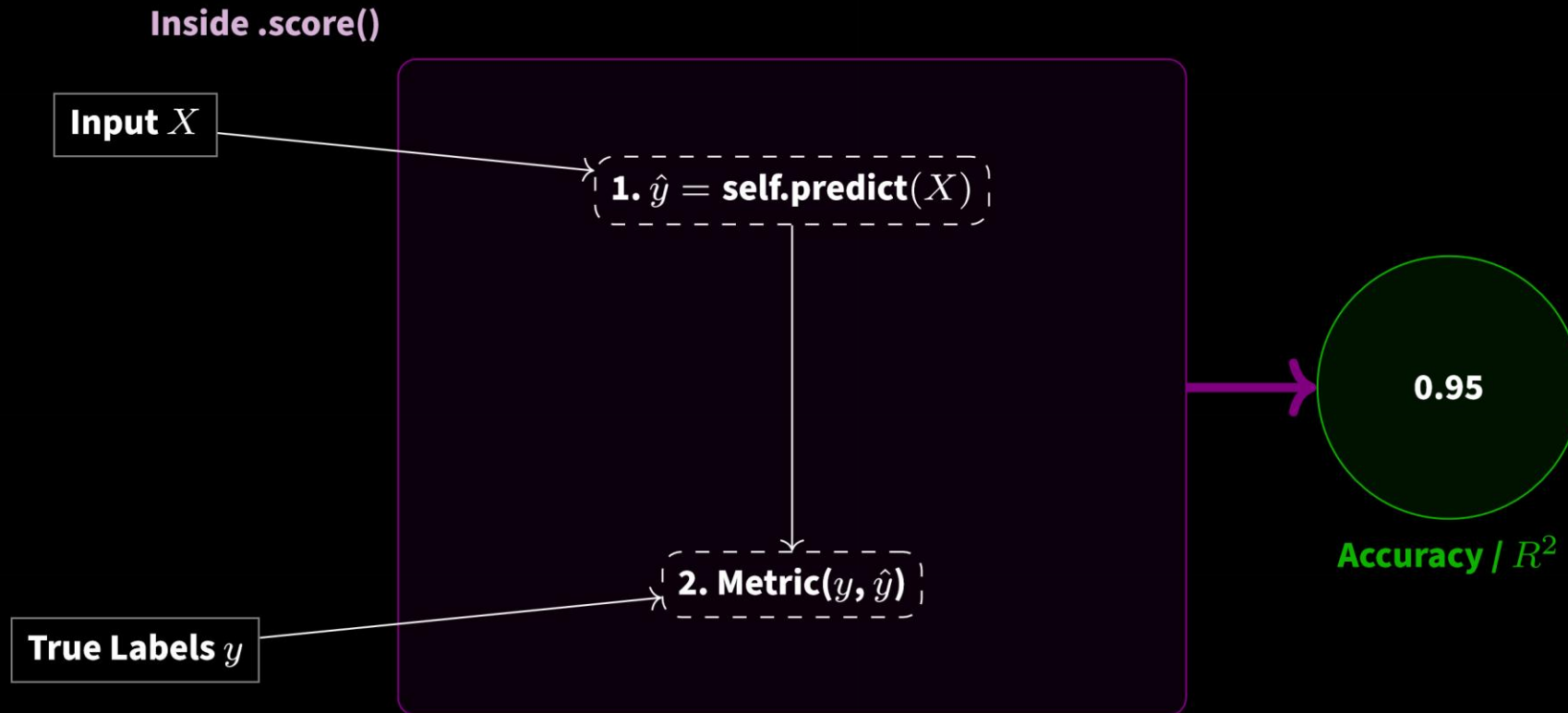
### 3.2 The Golden Rule: The Iron Wall

The most critical concept in ML lifecycle is knowing *where* to apply each method. **Never call `.fit()` on your Test Set!** This causes data leakage.



### 3.3 Inside `.score(X, y)`

The score method is a wrapper. It quietly generates predictions and then compares them to the true answers.

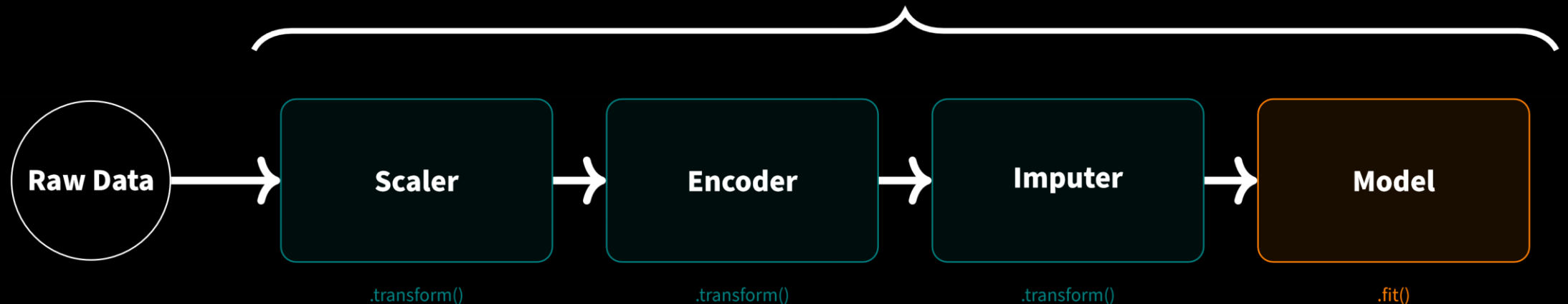


## Why Consistency Matters

### 4.1 Composability: The Lego Principle

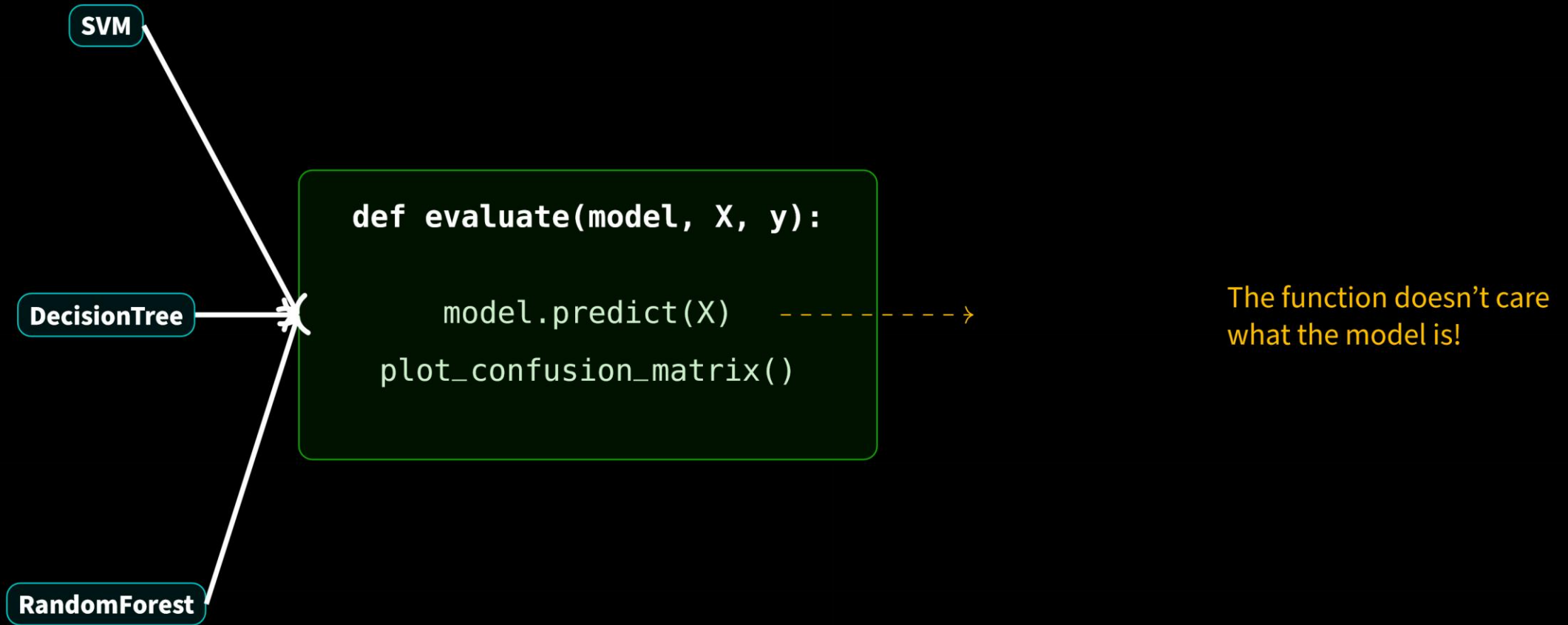
Because every Transformer outputs a matrix ( $X$ ) and accepts a matrix ( $X$ ), we can chain them infinitely. The output of one becomes the input of the next without writing glue code.

## The Pipeline



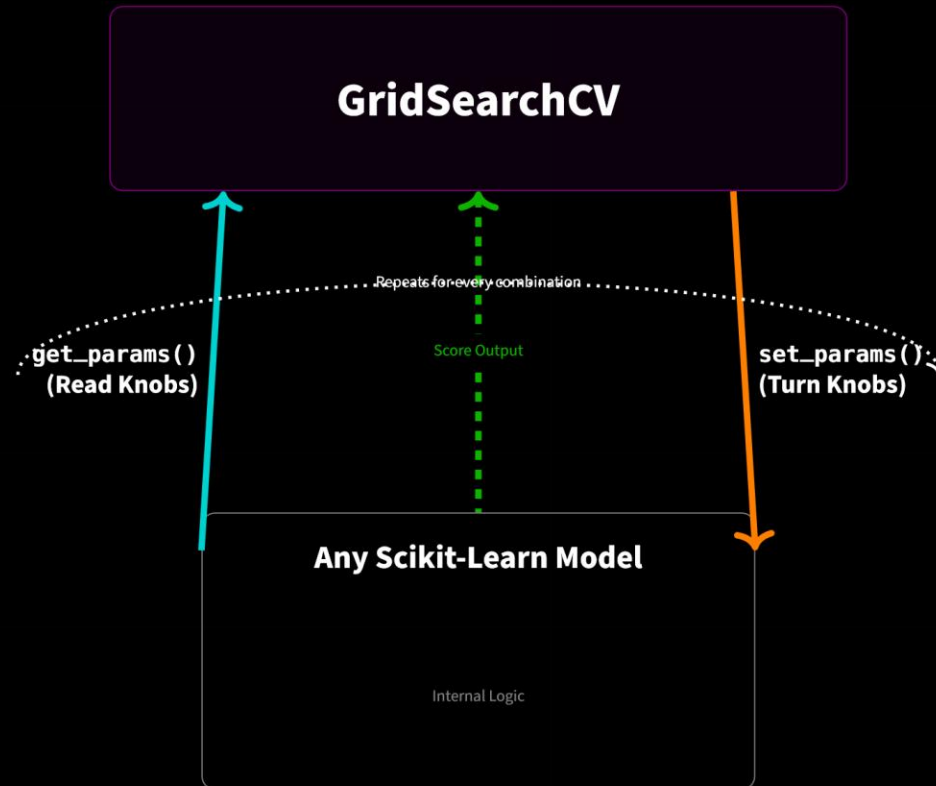
## 4.2 Reuse: Write Once, Run Everywhere

You can write a complex evaluation function (calculating precision, recall, ROC curves) **once**, and it will work for any classifier ever invented.



### 4.3 Search: Automated Optimization

How does GridSearchCV tune a model it has never seen before? It relies on the standard attribute `get_params()` to read the knobs, and `set_params()` to turn them.

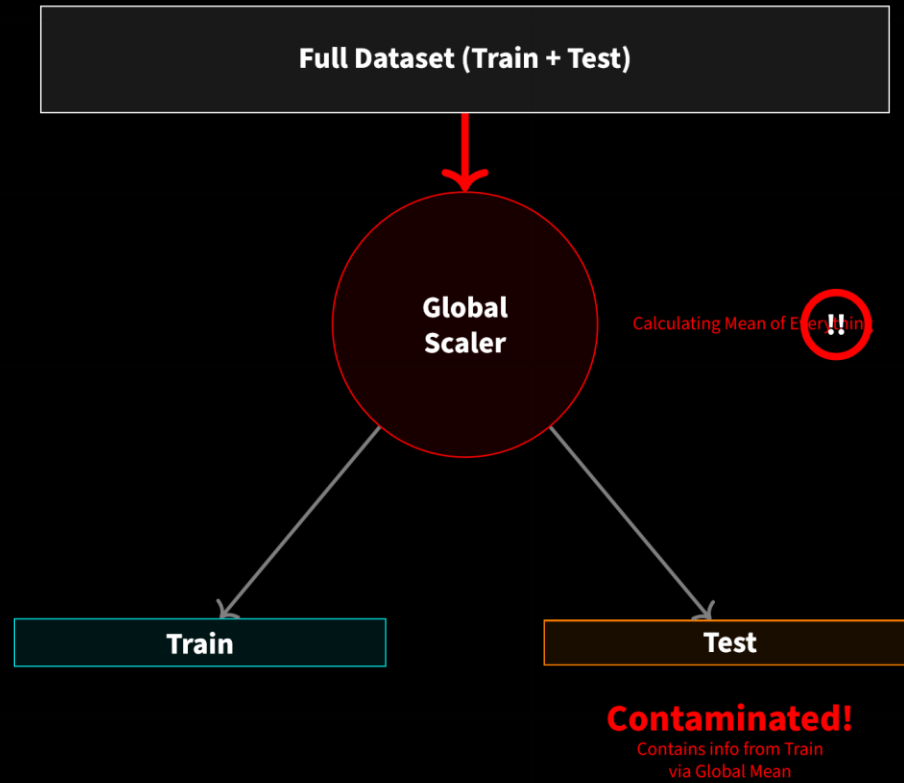


## Pipelines and Avoiding Data Leakage

### 5.1 The Silent Killer: Data Leakage

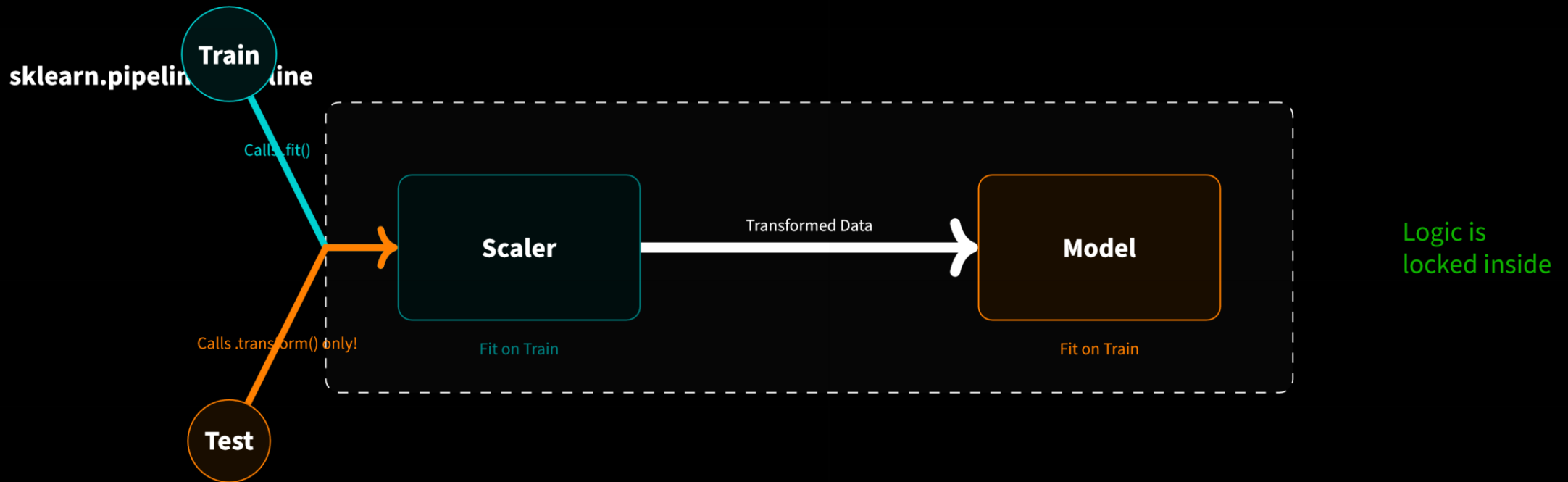
Data leakage occurs when information from outside the training dataset (like the test set) is used to create the model. **Common Culprit:** Scaling your data *before* splitting it. If you calculate the mean of the *entire* dataset, your model "knows" about the test set distribution.





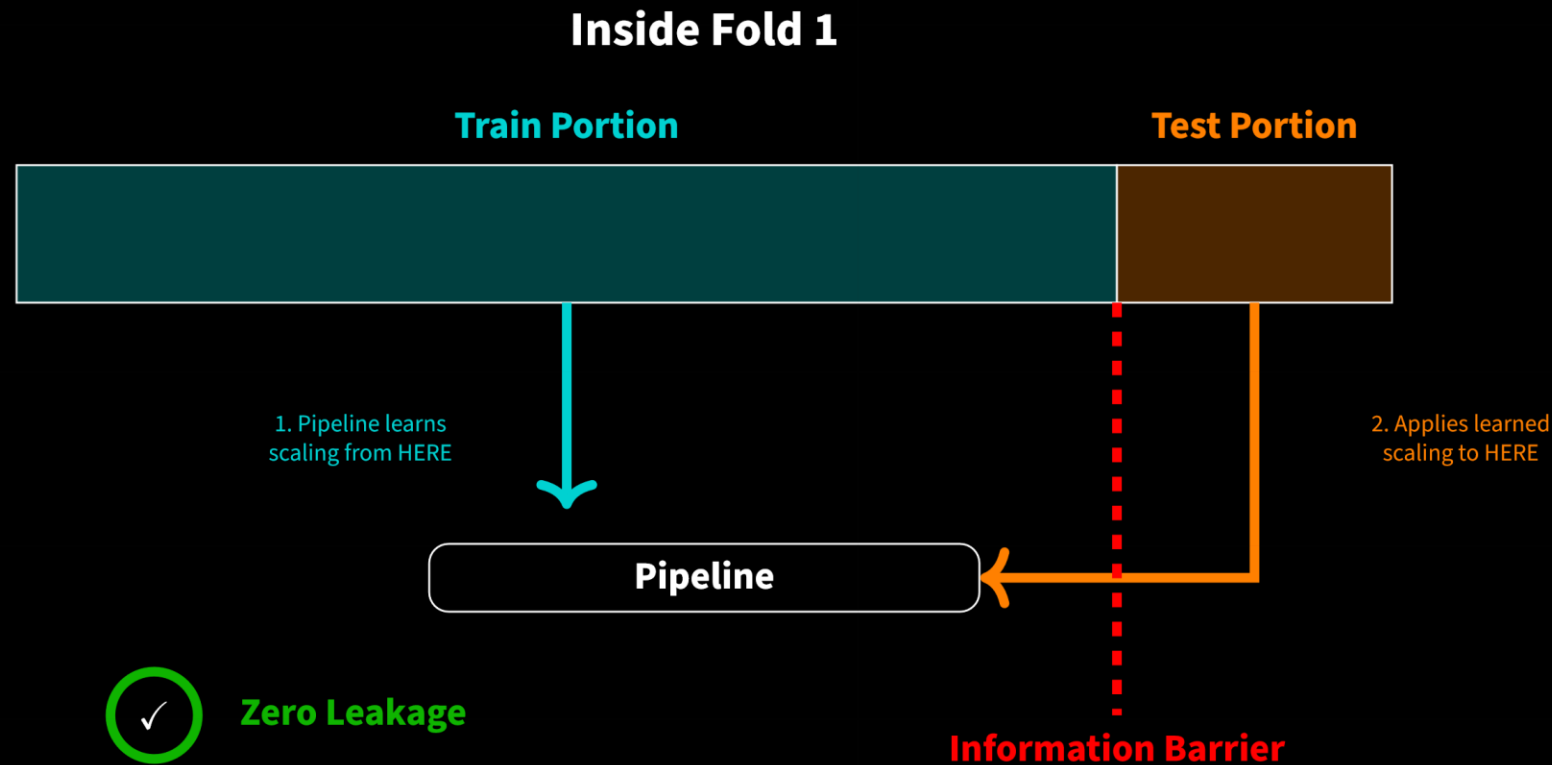
## 5.2 The Solution: The Pipeline Shield

A Pipeline bundles preprocessing and modeling into a single object. Crucially, it ensures that `fit` is only applied to the training data, and that learned parameters (like mean/std) are merely *applied* to the test data.



### 5.3 Safety in Cross-Validation

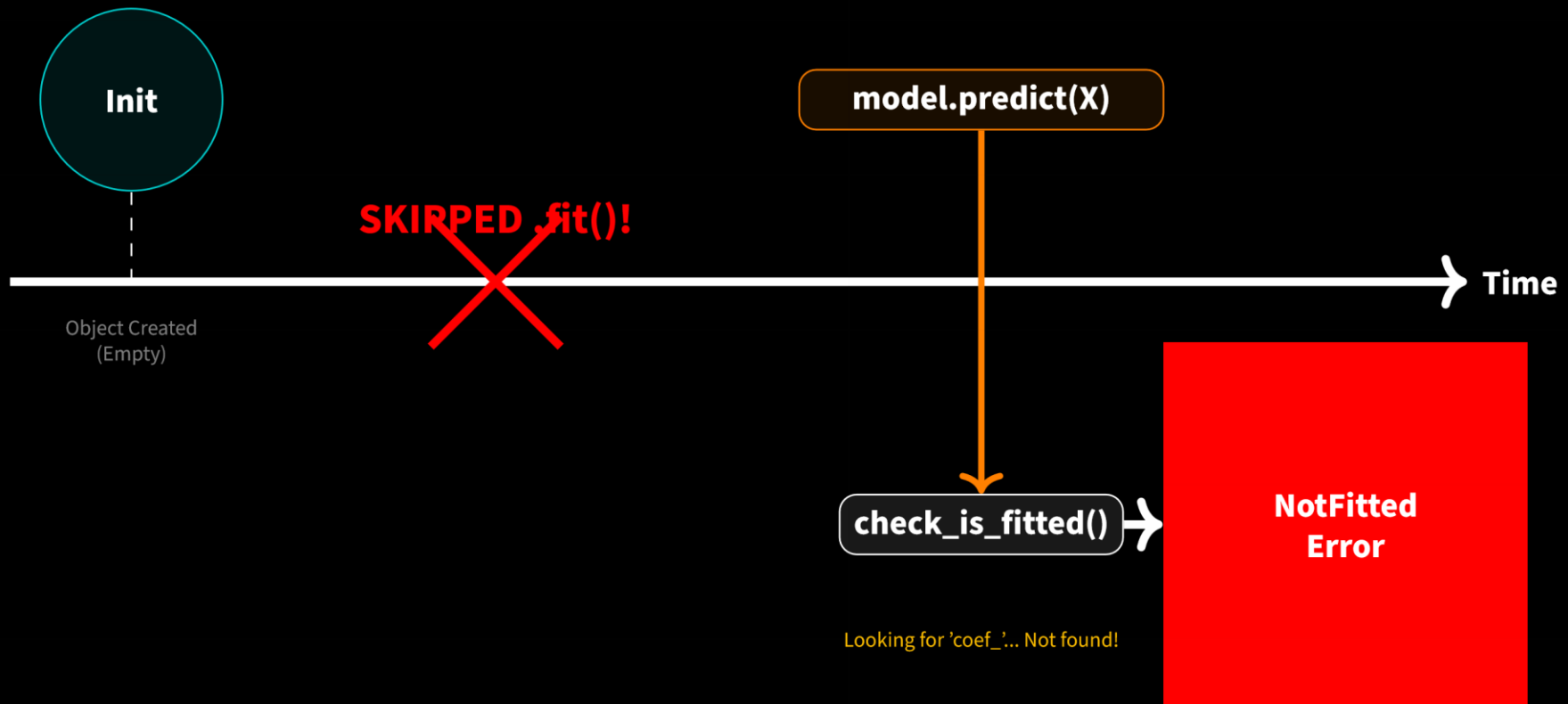
When you use `cross_val_score(pipeline, X, y)`, scikit-learn automatically splits the data *first*, then fits the pipeline on the training fold. This makes leakage mathematically impossible.



## Common Pitfalls

### 6.1 The Premature Launch: `NotFittedError`

This error occurs when you try to use a model (predict/transform) before teaching it (fit). Internally, scikit-learn checks for the presence of attributes ending in `_` (like `coef_`). If they are missing, it raises the alarm.



## 6.2 The Memory Wipe: Mixing fit and transform

A common mistake is re-fitting on the test set: `scaler.fit(X_test)`. **Why it fails:** `fit()` is destructive. It overwrites the previously learned parameters (mean/std from training) with new ones from the test set, destroying your calibration.

### 1. Correct State

```
mean_ = 50  
scale_ = 10
```

Learned from Train

**scaler.fit  
(X\_test)**

### 2. Corrupted State

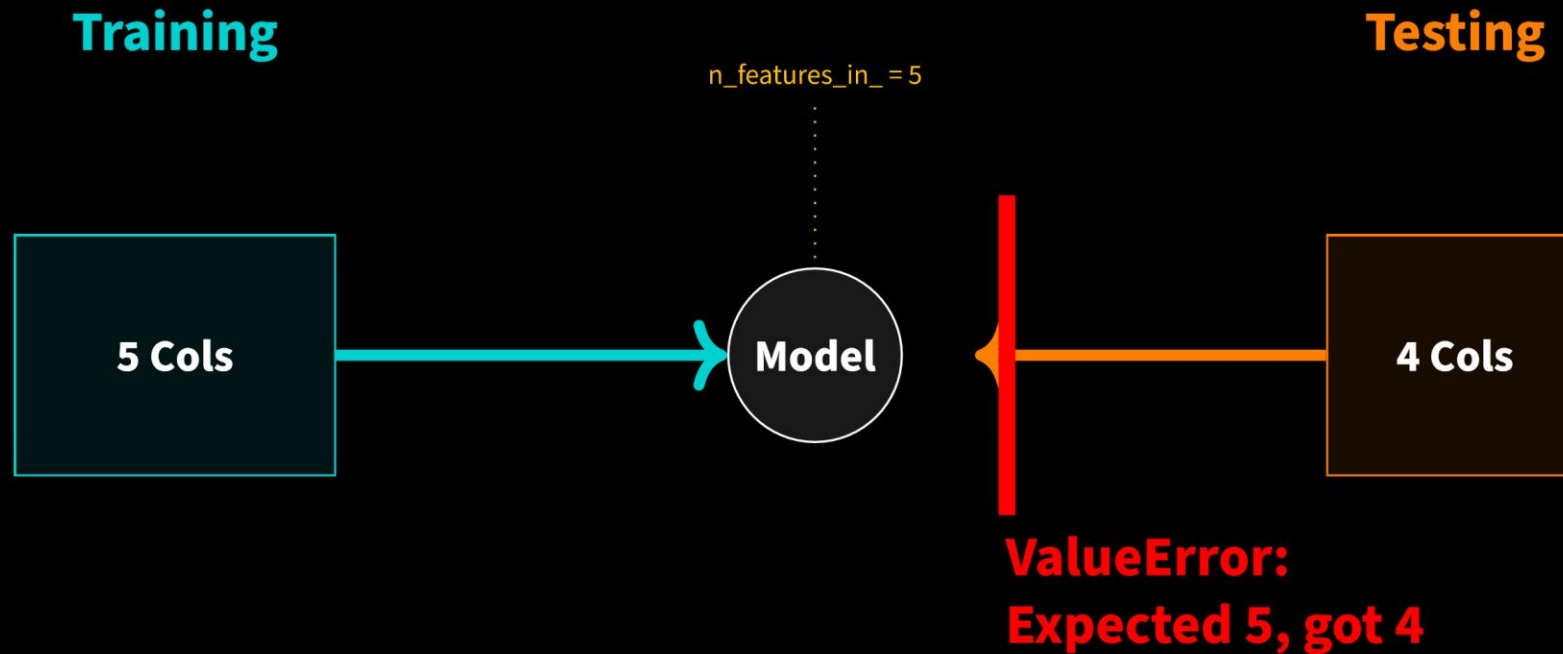
```
mean_ = 52  
scale_ = 11
```

Learned from Test (WRONG!)

The old memory is gone forever.  
Use `.transform()` instead!

## 6.3 The Dimension Mismatch

Scikit-learn remembers the shape of your training data. If you fit on 5 columns, you cannot predict on 4 columns. The "Shape Contract" is strict.

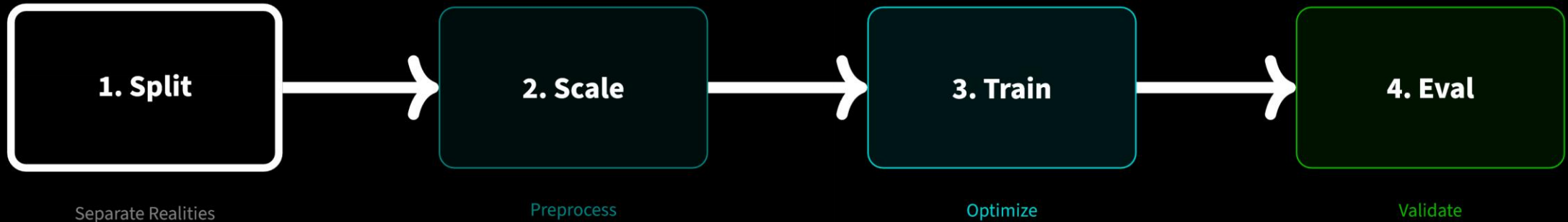


### Mini Walkthrough: The Full Workflow



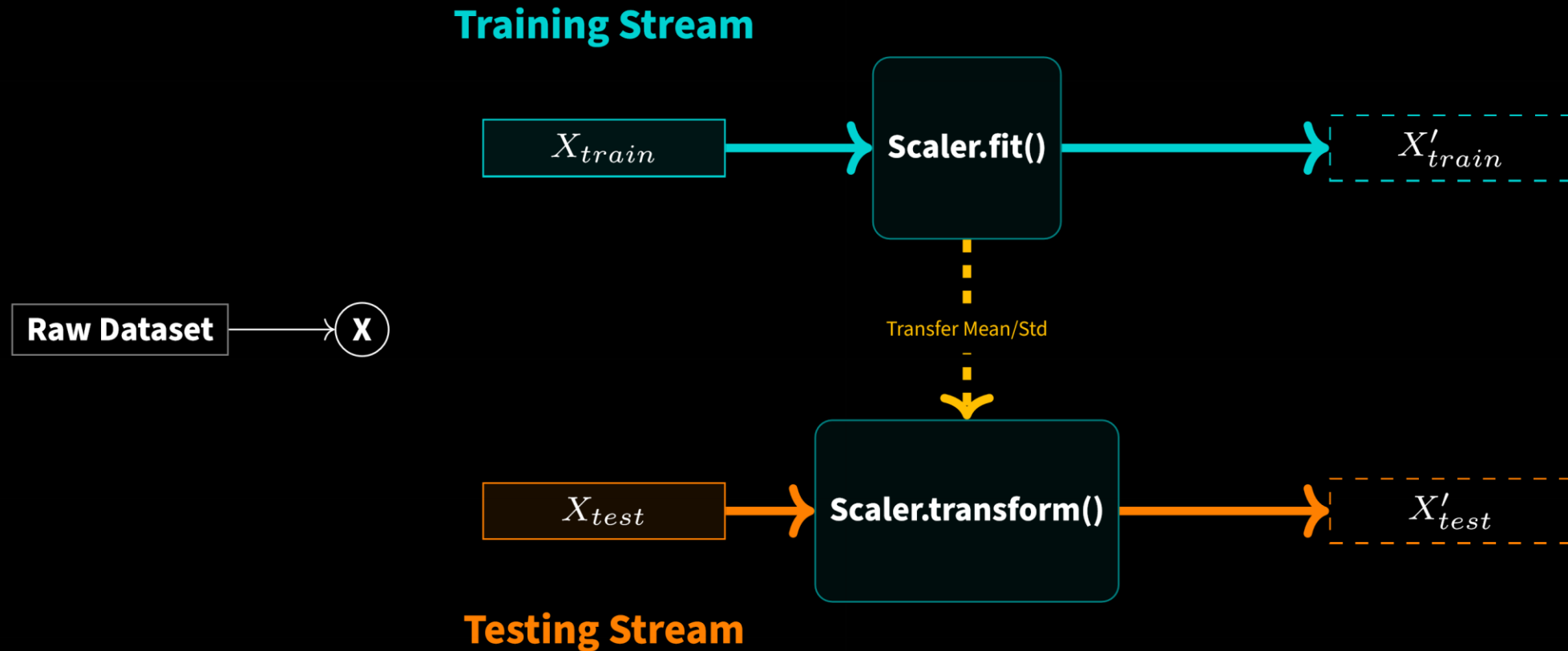
## 7.1 The Roadmap

We will follow a 4-step process. This specific order is mandatory to ensure mathematical correctness and avoid data leakage.



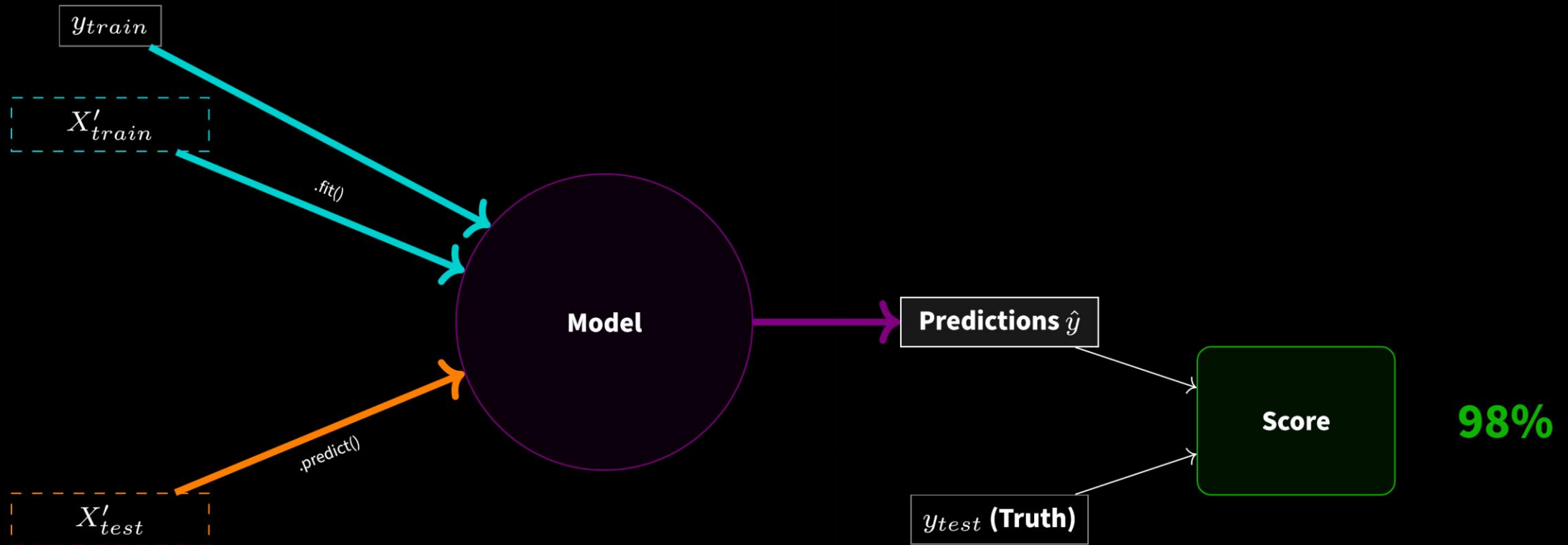
## 7.2 Split & Scale (Avoiding Leakage)

The Scaler must "learn" the mean and variance **only** from the Training set, but must apply that transformation to **both** sets.



### 7.3 Train & Evaluate

The Model learns from the *Clean* training data. Then, it tries to predict the *Clean* test data. Finally, we compare those predictions to the answers.



## 7.4 The Code Translation

Mapping the concepts to the actual Python syntax.

### workflow.py

```
# 1. Split
X_tr, X_te... = train_test_split(X, y)
# 2. Scale (Watch closely!)
scaler = StandardScaler()
X_tr = scaler.fit_transform(X_tr)
X_te = scaler.transform(X_te)

# 3. Train
model = LogisticRegression()
model.fit(X_tr, y_tr)
# 4. Evaluate
acc = model.score(X_te, y_te)
```

Never fit on test!

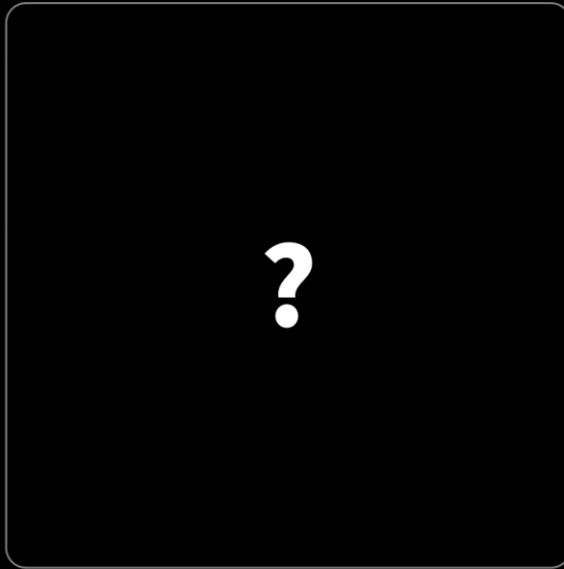
## Where We're Headed Next

### 8.1 Opening the Black Box

So far, we have been **Drivers** of the car. We know the steering wheel (fit) and the pedals (predict).

In the next lecture, we become **Mechanics**. We will pop the hood to understand exactly how the engine is built.

Current View

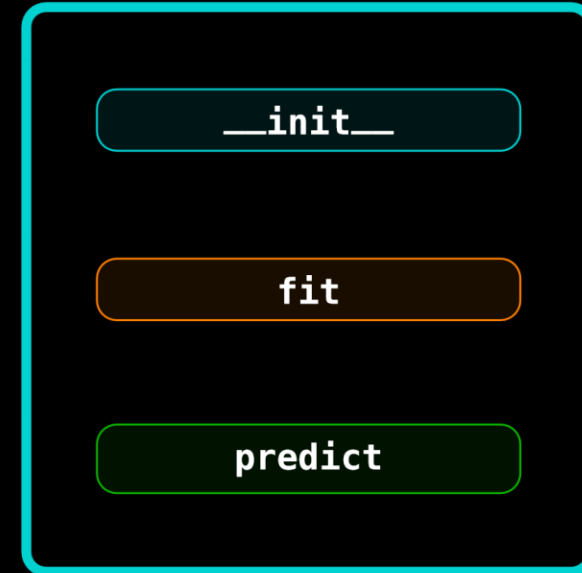


The "Magic" Box

Deep Dive



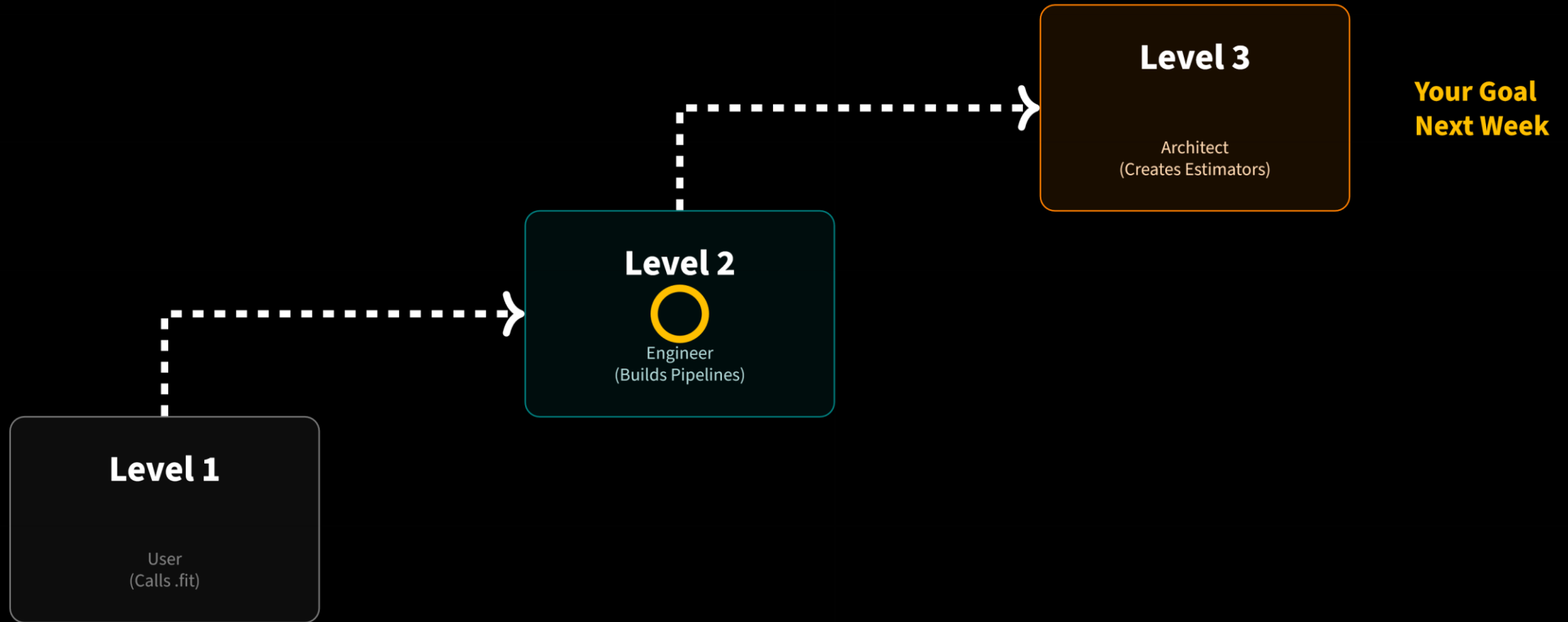
Next Lecture



The Source Code

## 8.2 From Consumer to Architect

Why learn the anatomy? Because standard models aren't enough for cutting-edge research. To implement your own algorithms and have them work with GridSearch, you must master the **Estimator Contract**.





### 8.3 Sneak Peek: The Blueprint

We will write a class from scratch. We will define the rules of how parameters are stored and how data is processed.

```
class CustomModel(BaseEstimator):  
  
    def __init__(self, ...):  
        # Store Config  
  
    def fit(self, X, y):  
        # The Math Logic  
        return self
```

**COMING SOON**

## **End of Lecture 01**

Intelligence Academy